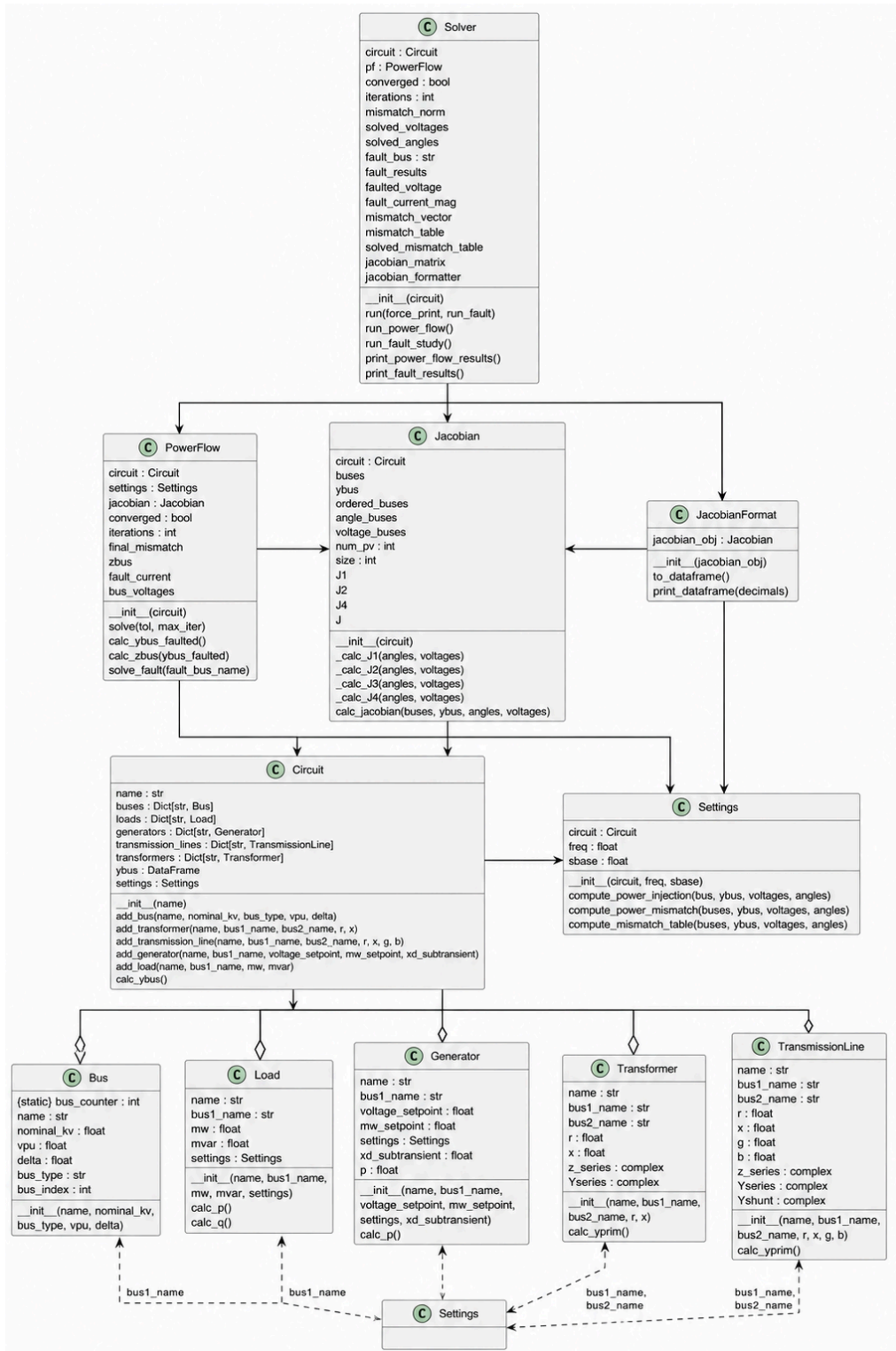


Project 2 Documentation



Bus Class:

The Bus class represents a node in our power system where different components connect. Each bus has a name and nominal voltage level in kilovolts, which defines its rated operating voltage. The class will automatically assign a unique index to each bus using a counter, allowing each bus to be identified in the network. Each bus also stores its type (Slack, PQ, or PV), voltage magnitude (vpu), and voltage angle (delta). A unique bus_index is assigned to each bus using a counter so it can be referenced correctly in matrix calculations.

Transformer Class:

The Transformer class represents a transformer that connects two buses in the power system. It will store the transformer name, the name of the two connected buses, and its series resistance and reactance. This class models how voltage levels will be linked between buses and how the impedance affects power flow between them. The calc_yprim() method builds the primitive admittance matrix for the transformer, which is used when constructing the system Ybus matrix.

Transmission Line Class:

The transmission line class represents a transmission line connecting two buses in the power system. It stores the line's name, the name of the two connected buses, and the electrical parameters: resistance, reactance, conductance, and susceptance. These parameters define the impedance and admittance characteristics of the line to model real and reactive power flow between buses. The calc_yprim() method builds the primitive admittance matrix, including both the series admittance and the shunt admittance, which is split evenly between the two connected buses.

Load Class:

The load class represents an electrical load connected to a specific bus in the power system. It stores the load's name, the name of the bus it is connected to, its real power (MW) and reactive power (MVar). This class models the power consumption at a bus, which will help for power flow calculations. The class also stores system settings and uses the calc_p() and calc_q() methods to convert real and reactive power values into per-unit form.

Generator Class:

The generator class represents a power generator connected to a specific bus in the system. It stores the generator's name, the name of the bus it is connected to, its set voltage, and its real power output (MW). This class will model a power source that regulates voltage and supplies real power to a system. The class stores system settings and subtransient reactance (xd_subtransient). The calc_p() method converts real power into per-unit form, and xd_subtransient is used later for fault analysis calculations.

Circuit Class:

The Circuit class represents the entire power system network. It stores all buses, loads, generators, transformers, and transmission lines in dictionaries using their names as keys. This allows each component to be accessed and managed easily. The class provides methods to add each type of component while checking for errors such as duplicate names or invalid bus connections.

The most important method in this class is calc_ybus(), which builds the system admittance matrix (Ybus). It loops through all transformers and transmission lines, calculates their primitive admittance matrices, and adds their contributions to the correct positions in the overall Ybus matrix. This matrix is used in all power flow and fault calculations.

Settings Class:

The compute_power_injection() method calculates the real and reactive power at a bus using the voltage magnitudes, bus angles, and the Ybus matrix. Inside the method, the code loops through every bus in the system and uses the real and imaginary parts of the admittance matrix to calculate P_i , Q_i , P_{ij} , and Q_{ij} . The bus angles are converted from degrees to radians before using the trig functions. This method returns the calculated real and reactive power for the selected bus.

Complex power equation:

$$S_i = V_i \sum_{j=1}^n Y_{ij}^* Y_j^*$$

Real power equation:

$$P_i = |V_i| \sum_{j=1}^n |V_j| (G_{ij} \cos(\delta_i - \delta_j) + B_{ij} \sin(\delta_i - \delta_j))$$

Reactive power equation:

$$P_i = |V_i| \sum_{j=1}^n |V_j| (G_{ij} \cos(\delta_i - \delta_j) - \beta_{ij} \sin(\delta_i - \delta_j))$$

Mismatch calculation

The `compute_power_mismatch()` method builds the mismatch vector by comparing specified power values to calculated power values. For each bus except the slack bus, the code finds the calculated injection values first, then determines the specified values by checking connected generators and loads. Generator real power is added, while load real and reactive powers are subtracted. After that, the mismatch is found by subtracting calculated power from specified power.

Real power mismatch:

$$\Delta P_i = P_i^{spec} - P_i^{calc}$$

Reactive power mismatch:

$$\Delta Q_i = Q_i^{spec} - Q_i^{calc}$$

For bus handling:

- **Slack bus**
 - Skipped in the mismatch vector
- **PV bus**
 - Includes only real power mismatch
- **PQ bus**
 - Includes both real and reactive power mismatch

Mismatch table

The `compute_mismatch_table()` method was also added to make the results easier to read. This method calculates the mismatch at every bus and puts the results into a pandas DataFrame. The table includes the bus number, name, type, and mismatch values in MW, Mvar, and MVA. For PV buses, reactive mismatch is set to zero, and for the slack bus both mismatches are set to zero. The apparent mismatch is found using the magnitude of the real and reactive mismatch values.

Jacobian Class:

The Jacobian class constructs the Jacobian matrix used in the Newton-Raphson power flow method. It organizes buses into groups based on their type. Slack buses are excluded because their voltage and angle are fixed. PV buses are included in the angle portion but excluded from the voltage magnitude portion since their voltage is fixed.

The class calculates four submatrices:

$$J1 \rightarrow \partial P / \partial \delta$$

$$J2 \rightarrow \partial P / \partial V$$

$$J3 \rightarrow \partial Q / \partial \delta$$

$$J4 \rightarrow \partial Q / \partial V$$

Each submatrix represents how changes in voltage magnitude or angle affect real and reactive power. These matrices are then combined into one full Jacobian matrix, which is used to solve for updates in voltage and angle during each iteration of the Newton-Raphson method.

Powerflow Class:

The PowerFlow class performs the Newton-Raphson power flow solution. It begins by building the Ybus matrix and initializing voltage magnitudes and angles.

During each iteration, it:

- Computes the mismatch vector
- Checks if the solution has converged
- Builds the Jacobian matrix
- Solves for updates to voltage magnitudes and angles

These updates are applied to all non-slack buses, and the process repeats until the mismatch is below the tolerance. If the solution converges, the final voltages and angles are stored back into the bus objects.

This class also includes fault analysis methods. It modifies the Ybus matrix to include generator subtransient reactances, computes the Zbus matrix, calculates the fault current at a selected bus, and determines the post-fault voltages at all buses. The faulted Ybus is built by adding each generator's subtransient admittance to the diagonal entry of its terminal bus, and inverting it gives the Zbus, whose diagonal element represents the Thevenin impedance seen at the faulted bus. The fault current is then found by dividing the prefault voltage by the sum of this Thevenin impedance and any specified fault impedance, assuming a balanced three-phase fault. Post-fault voltages at every bus are recovered from the Zbus mutual terms, showing how the disturbance propagates through the network and depresses voltages most strongly at buses electrically close to the fault.

Solver Class:

The Solver class controls the overall execution of the program. It initializes the power flow object and runs the full solution process.

It first calculates the flat-start mismatch and Jacobian matrix to verify the system setup. It then runs the Newton-Raphson power flow solution and stores the results, including the number of iterations, convergence status, and final mismatch.

If fault analysis is enabled, the Solver class runs the fault study and stores the fault current and post-fault voltages.

The class also includes print functions that display:

- Initial mismatch table
- Jacobian matrix
- Final bus voltages and angles
- Final mismatch results
- Optional fault study results